



← Back to student guides

✦ Airflow

Orchestration

3 levels

113 sections



The Complete Guide

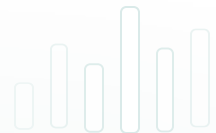
Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

DAG

SCHEDULER

EXECUTOR

WORKER



Part one of three. A fast review of **everything in the Beginner Detailed track**, in about twenty-five minutes. Fast review first, common questions at the end. Mid-level reviews this plus its own material; Senior reviews all three.

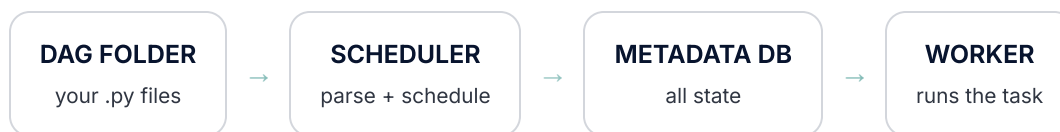




Airflow is a platform for writing, scheduling, and monitoring workflows as Python code. You define a DAG, a directed acyclic graph of tasks, and Airflow works out the execution order, runs it on a schedule, retries failures, and records every run in a metadata database with a UI on top. It replaces a chain of cron jobs that had no dependency graph, no retries, and no history.

Then add the sentence that shows you have used it: "the thing people get wrong is that Airflow is an orchestrator, not a compute engine: a task should tell Spark or the warehouse to do the work, not load a dataframe into the worker."

The architecture



Component	Job	You notice it when
Scheduler	Parses DAG files, decides what is ready, queues it	Nothing runs, or runs late
Executor	Hands queued tasks to workers	Tasks stuck in <code>queued</code>
Worker	Runs your task code	The task fails or runs out of memory
Metadata DB	Every run, task instance, connection, variable	The UI is slow; state looks wrong
Webserver	UI and REST API	You cannot see anything, but runs continue
Triggerer	Holds deferred tasks asynchronously	Deferred tasks hang if it is not running



not your file. The file says what should exist, the database records what happened.

Vocabulary

Term	Say this
DAG	A directed acyclic graph of tasks: the workflow definition
Operator	A class describing a kind of work: <code>BashOperator</code> , <code>PythonOperator</code>
Task	An instance of an operator in a DAG, identified by <code>task_id</code>
Task instance	One run of that task for one specific interval
DAG run	One execution of the whole DAG for one interval
Logical date	The interval the run covers: not the wall clock
XCom	Cross-communication: small values passed between tasks
Connection	Stored credentials and host details for an external system
Variable	A stored non-secret configuration value
Pool	A cap on concurrent tasks across all DAGs, protecting a resource
Sensor	An operator that waits for an external condition

Your first DAG, annotated



```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.empty import EmptyOperator

with DAG(
    dag_id="hello_world",          # unique; renaming abandons its history
    start_date=datetime(2024, 1, 1), # the first interval considered
    schedule="@daily",             # cron, preset, timedelta, or None
    catchup=False,                 # do NOT run every missed interval
    default_args={"retries": 3, "retry_delay": timedelta(minutes=5)},
    tags=["tutorial"],
) as dag:
    start = EmptyOperator(task_id="start")
    hello = BashOperator(task_id="hello", bash_command="echo hi {{ ds }}")
    start >> hello                  # ">>" means "then"
```

Mandatory: `dag_id` , `start_date` , and per task a `task_id` . Everything else has a default, and two of those defaults will hurt you, which is why `catchup=False` and explicit retries appear in every example.

TaskFlow versus classic operators

PYTHON

Copy

```
@dag(dag_id="etl", start_date=datetime(2024, 1, 1), schedule="@daily", catchup=False)
def etl():
    @task
    def extract(): return {"rows": [1, 2, 3]}

    @task
    def transform(d: dict): return {"total": sum(d["rows"])}

    @task
    def load(r: dict): print(r["total"])
```



✓ TaskFlow: for Python work

- Dependencies inferred from the function calls
- Return values passed automatically via XCom
- Normal signatures and type hints

✗ Classic operators: for systems

- Explicit `>>` dependencies
- Manual `xcom_push` / `xcom_pull`
- The only option for non-Python operators

They mix freely. Use TaskFlow for Python you wrote; use a provider operator for "run this SQL" or "submit this Spark job".

Scheduling: the interval model

Airflow schedules intervals, not moments. A `@daily` run for 1 May starts just after midnight on 2 May, because the data for 1 May is only complete once 1 May is over.



05-01 00:00

Interval starts

`data_interval_start` . Nothing runs yet.



05-02 00:00

Interval ends

The run is now scheduled.



05-02 00:00:05

Run starts

Logical date is **2024-05-01**, the interval rather than the clock.

Parameter	Common mistake
<code>start_date</code>	Setting it to today and wondering why nothing ran
<code>schedule</code>	Expecting it to fire <code>at</code> the start of the interval



catchup

Leaving it `true` with an old start date. Hundreds of runs at once

Use `{{ ds }}` , never `datetime.now()`

A task that computes "yesterday" from the wall clock produces different results on a rerun than it did originally, which destroys reproducibility and makes backfills meaningless. The interval is available as a template variable for exactly this reason.

Template variable	Value for the 1 May interval
<code>{{ ds }}</code>	<code>2024-05-01</code>
<code>{{ ds_nodash }}</code>	<code>20240501</code>
<code>{{ data_interval_start }}</code> / <code>{{ data_interval_end }}</code>	Interval bounds
<code>{{ prev_ds }}</code> / <code>{{ next_ds }}</code>	Neighbouring intervals
<code>{{ dag_run.run_id }}</code> / <code>{{ ts }}</code>	Run identifier, full timestamp
<code>{{ macros.ds_add(ds, -7) }}</code>	Date arithmetic without imports

Retries, timeouts, and idempotence

PYTHON

Copy

```
default_args = {
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
    "execution_timeout": timedelta(minutes=45),    # kill a hung task
}

@dag(..., default_args=default_args,
```



Parameter	Scope	Prevents
<code>retries</code> / <code>retry_delay</code>	Task	A transient blip failing the pipeline
<code>execution_timeout</code>	Task	A hung task holding a worker forever
<code>dagrun_timeout</code>	DAG	Intervals piling up behind a stuck run
<code>max_active_runs</code>	DAG	Two runs writing the same table
<code>depends_on_past</code>	Task	Running before the previous interval succeeded

Retries are only safe if the task is idempotent

Running it twice must equal running it once. A task that `INSERT`s duplicates rows on retry; a task that `DELETE`s the target partition then inserts does not. Design for reruns and retries become free. Otherwise every retry is a data-quality incident.

sql/rollup.sql

Copy

```
-- Idempotent by construction: safe to retry, safe to backfill.
DELETE FROM daily_stats WHERE day = '{{ ds }}';
INSERT INTO daily_stats SELECT '{{ ds }}', count(*) FROM events
WHERE event_date = '{{ ds }}';
```

XCom

PYTHON

Copy

```
@task
def count() → int: return 4213 # pushed automatically
```



```
report(count())
```

✓ Pass through XCom

- A row count or checksum
- An S3 key or table partition
- A model version or run id

✗ Pass through storage instead

- A dataframe of any size
- A file's contents or query results
- Anything you would not put in a database cell

XCom is serialised into the metadata database. That is the whole reason for the rule: bloating it slows every UI page and the scheduler loop. Write the data to storage and pass the **path**.

Connections and Variables

BASH

Copy

```
airflow connections add warehouse --conn-type postgres \  
--conn-host db.internal --conn-login airflow --conn-password secret
```

PYTHON

Copy

```
SQLExecuteQueryOperator(task_id="rollup", conn_id="warehouse", sql="sql/rollup.sql")  
  
@task  
def count():  
    return PostgresHook(postgres_conn_id="warehouse").get_first("SELECT count(*) FROM ev
```

Store	For	Read with
Connection	Host, port, credentials for a system	<code>conn_id=</code> on an operator, or a Hook



variable

Non-secret configuration

`variable.get()` , or `{{ var.value.x }}`

Never call `Variable.get()` at the top level

Top-level code runs on every parse, every thirty seconds, per file. One misplaced `Variable.get()` is 2,880 database queries a day for a value nobody read. Use it inside a task, or the template form `{{ var.value.x }}` , which resolves at run time.

Sensors

PYTHON

Copy

```
S3KeySensor(
    task_id="wait",
    bucket_key="events/{{ ds }}/_SUCCESS",
    poke_interval=60,
    timeout=60 * 60 * 4,          # ALWAYS set a timeout
    mode="reschedule",          # ← releases the worker slot between checks
)
```

Mode	Worker slots while waiting
<code>poke</code> (default)	One, continuously: can deadlock a cluster
<code>reschedule</code>	Zero between checks

Sensors in `poke` mode can deadlock your cluster

Ten sensors waiting four hours occupy ten worker slots for four hours, possibly including the slots needed by the upstream tasks they are waiting for. Use `reschedule` for anything waiting more than a minute or two, and always set a `timeout` . Mid level replaces sensors with deferrable operators entirely.



PYTHON

Copy

```
@task.branch
def choose(count: int) → str:
    return "full_reload" if count > 100_000 else "incremental_load"

join = EmptyOperator(
    task_id="join",
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,  # ← required
)
```

Trigger rule	Runs when
<code>all_success</code> (default)	Every upstream succeeded
<code>all_done</code>	Every upstream finished, whatever the result: cleanup tasks
<code>one_success</code> / <code>one_failed</code>	At least one succeeded / failed
<code>none_failed</code>	Nothing failed; skips are acceptable
<code>none_failed_min_one_success</code>	The branch-join rule



Graph	what happened across the last N runs, live here
Graph	The dependency structure, and where it stopped
Gantt	Which task is the bottleneck
Code	What the scheduler is running
Rendered Template	What your <code>{{ ds }}</code> became
XCom	What the task passed downstream
Admin → DAG Processing	Why the scheduler is slow: parse times

State	Means
<code>success</code> / <code>failed</code>	Done / failed after retries
<code>up_for_retry</code>	Failed, will try again
<code>up_for_reschedule</code>	A sensor waiting in reschedule mode
<code>queued</code>	Waiting for a worker slot
<code>skipped</code>	Skipped by a branch or trigger rule
<code>upstream_failed</code>	Never ran; an upstream failed

"Clear" is the verb, not "rerun". You clear a task instance's state and the scheduler notices there is now a stateless task instance to run. That indirection explains why clearing works on old runs and can cascade downstream.



✓ **Inside a task**

- Database queries and API calls
- `Variable.get()`
- Reading files
- Heavy imports: pandas, torch

✗ **At the top level**

- Only the DAG definition and task wiring
- Only cheap constants and light imports
- Runs every 30 seconds, per file, forever

The one-line test: ask of every line outside a task: "do I want this to run every thirty seconds forever?" If no, it belongs inside a task.

Common interview questions

01

What is Airflow and what problem does it solve?

A platform for writing, scheduling, and monitoring workflows as Python code. It gives you a dependency graph, retries, a scheduling model with historical intervals, and a UI with full run



02

What is a DAG?

A directed acyclic graph of tasks. Directed because dependencies point one way; acyclic because a task cannot eventually depend on itself, which Airflow enforces as an import error. That guarantee is what lets the scheduler always compute an execution order.

03

Name the components and what each does.

Scheduler parses DAG files and decides what is ready. Executor hands queued tasks to workers. Workers run the code. The metadata database holds all state. The webserver serves the UI and API. The triggerer holds deferred tasks. The webserver going down does not stop pipelines running.

04

Operator versus task versus task instance?

The operator is the class, a kind of work. The task is an instance of it in a DAG with a `task_id`. The task instance is one run of that task for one specific interval. The UI shows task instances; your file defines tasks.

05

Why does a `@daily` DAG for 1 May run on 2 May?

Airflow schedules intervals, not moments, and a run fires at the `end` of its interval, because the data for 1 May is only complete once 1 May is over. The logical date is 2024-05-01 even though the wall clock says the second.

06

What does `catchup=True` do, and why is it dangerous?

It creates a run for every interval between `start_date` and now. With a two-year-old start date and a daily schedule that is around eight hundred runs the moment you unpause, which can saturate your cluster and hammer every downstream system. Default to `False` and backfill deliberately.

07

Why use `{{ ds }}` instead of `datetime.now()` ?

Because a rerun or a backfill must produce the same result as the original run. `now()` makes the output depend on when you ran it, so clearing an old task instance silently reprocesses the wrong window.



database, so row counts, paths, and versions are fine while dataframes and file contents are not. Those bloat the database and slow every UI page. Write the data to storage and pass the path.

09

Where do credentials belong?

In a Connection, referenced by `conn_id` from an operator or a Hook. Never in the DAG file, and never in a Variable you print. Non-secret configuration goes in a Variable.

10

Why must you not call `Variable.get()` at the top level?

Top-level code executes on every parse cycle (every thirty seconds, for every DAG file) so it becomes thousands of database queries a day for a value nobody read. Worse, an API call up there adds its latency to every parse and can stall the scheduler.

11

What makes a task idempotent, and why does it matter?

Running it twice produces the same result as once. It matters because Airflow retries, and because you will eventually backfill. The standard pattern is deleting the target partition then writing it, rather than appending.

12

What is a sensor, and what is the `mode` trap?

An operator that waits for an external condition. In the default `poke` mode it holds a worker slot for the entire wait, so a handful of long waits can deadlock a cluster.

`mode="reschedule"` releases the slot between checks, and always set a `timeout`.

13

How do you make one branch of a DAG run conditionally?

A branching operator whose callable returns the `task_id` to follow; every other branch is skipped. The catch is that any join afterwards needs an explicit trigger rule, usually

`none_failed_min_one_success`. Otherwise the skip propagates and silently skips the rest of the DAG.

14

What trigger rule would you use for a cleanup task?

`all_done`, so it runs whether the upstream work succeeded, failed, or was skipped. That is the case the default `all_success` cannot express.



state. There is no rerun button. Understanding that indirection explains why clearing works on historical runs and why

Clear + Downstream

cascades.

16

What is the difference between `execution_timeout` and `dagrun_timeout` ?

The first kills an individual task that runs too long; the second fails the whole run. You want both: the task timeout stops a hang holding a worker slot, and the run timeout stops intervals piling up behind a stuck run.

17

How do you test a single task quickly?

`airflow tasks test dag_id task_id 2024-05-01` . It runs in the foreground, streams logs to your terminal, and records no state in the metadata database, which is the fastest feedback loop available.

18

How do you backfill, and what is the risk?

`airflow dags backfill` with a date range. The risk is that thirty days of a twenty-task DAG is six hundred task instances competing with live pipelines. Set `max_active_runs` , and only backfill something idempotent.

19

Why is Airflow not a good place to process data?

It is an orchestrator. A task that loads a large dataframe into the worker consumes a scheduling slot for the duration, cannot scale beyond one machine, and puts memory pressure on a shared component. The task should tell Spark, the warehouse, or a container to do the work.

20

Your DAG is not showing up in the UI. Debug it.

Is the file in the DAG folder with a `.py` extension; does it import at all, which `python dags/my_dag.py` answers; is there an entry in `airflow dags list-import-errors` ; does the file create a DAG object at module level; and has the scheduler had thirty seconds to notice.



- Give the thirty-second answer, then the "orchestrator, not compute engine" sentence.
- Name the six components and one symptom each.
- Say what the scheduler does to your DAG file, and how often.
- Explain why a `@daily` run for 1 May starts on 2 May.
- Say what `catchup=True` with an old `start_date` does.
- Give the reason to use `{{ ds }}` over `now()`.
- Say what XCom is for and what it must not carry.
- Name where credentials belong, and the top-level-code rule.
- Give the sensor mode that frees a worker slot, and the missing-timeout risk.
- Say why a join after a branch needs a trigger rule.
- Explain what "clear" does and why there is no rerun button.
- Define an idempotent task and give the SQL pattern for one.



PREVIOUS

Beginner · Detailed

NEXT UP

Beginner · Tips & Tricks





LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)

[Zomra](#)



Brainsmingle space

Free to join this week with code **FREECORETEAM**



MLOps MENA
COMMUNITY

العربية



Join the Community ↗

